Week 2 Lecture Notes: Variational Autoencoders

Hong Qin, Old Dominion University

Fall 2026

Audience: Graduate students learning Variational Autoencoders (VAEs) and variational inference. *Goal:* Understand what a latent-variable model is, how the ELBO is derived (two ways), why the signs are what they are, what KL means, how the reparameterization trick works, and how to train and evaluate a VAE in practice.

Contents

1 Topic 1: Latent-Variable Modeling and Representation Learning **1**
1.1 Plain-English summary . . . 1
1.2 From observations to a hidden cause . . . 1
1.3 Autoencoders: useful representations, not a generative model . . . 2
1.4 Representation learning, briefly . . . 2
1.5 Payoff . . . 3

2 Topic 2: Variational Inference and the ELBO **3**
2.1 Plain-English summary . . . 3
2.2 Overview of this topic . . . 3
2.3 Acronyms and notation . . . 3
2.4 Prerequisites (quick refresher) . . . 5
2.5 Kullback-Leibler (KL) divergence: definition, notation, intuition . . . 6
2.6 The VAE setting and objective . . . 6
2.7 ELBO derivation A (via Jensen's inequality) . . . 7
2.8 ELBO derivation B (KL decomposition / "gap" identity) . . . 7
2.9 How Eq. (2) becomes Eq. (3) in Kingma and Welling . . . 8
2.10 Reparameterization trick (why we can backprop through sampling) . . . 9
2.11 Sign conventions and what to optimize . . . 9
2.12 Key takeaways . . . 9

3 Topic 3: Reconstruction Loss, KL Divergence, and Sampling **10**
3.1 Plain-English summary . . . 10
3.2 From ELBO to a practical loss . . . 10
3.3 The KL term in closed form . . . 10
3.4 The reconstruction-vs-regularization tug-of-war, made concrete . . . 11
3.5 Sampling: how a trained VAE generates new data . . . 11
3.6 Latent-space structure: interpolation and arithmetic . . . 12
3.7 Payoff . . . 12

4 Topic 4: Practical Considerations When Training and Evaluating a VAE **12**
4.1 Plain-English summary . . . 12
4.2 Architecture choices . . . 12
4.3 Training knobs that matter . . . 13
4.4 Failure modes to recognize . . . 13
4.5 Evaluation: how do you know it worked? . . . 13
4.6 Compute footprint . . . 14
4.7 Payoff . . . 14

---

1 Topic 1: Latent-Variable Modeling and Representation Learning

1.1 Plain-English summary

Before we can talk about VAEs in Topic 2, we need to agree on what a *latent-variable* generative model is, and why a regular autoencoder is *not* one. This topic introduces the latent-variable framework, examines the gap between "compresses data well" (autoencoder) and "generates new data well" (generative model), and motivates the VAE as the smallest, most principled fix to that gap.

1.2 From observations to a hidden cause

Consider a dataset of images of handwritten digits. Each image is a long vector of pixel values, but intuitively the image is determined by a much smaller set of *hidden factors*: which digit was written, how thick the strokes are, how slanted the writing is, where on the canvas the digit sits. A latent-variable generative model takes this intuition literally: it posits a low-dimensional hidden variable z that captures these factors, and a stochastic mapping from z to the observed x.

Formally, we specify a simple prior $p(z)$ (typically $\mathcal{N}(0, I)$) and a decoder $p_\theta(x \mid z)$ implemented by a neural network. The joint distribution is

$$p_\theta(x, z) \;=\; p(z)\, p_\theta(x \mid z),$$

and the data distribution is the marginal

$$p_\theta(x) \;=\; \int p_\theta(x \mid z)\, p(z)\, dz.$$

Two things are now possible in principle.

- **Generation.** Sample $z \sim p(z)$, then sample $x \sim p_\theta(x \mid z)$.
- **Inference.** Given an observation x, compute the posterior over its latent cause,

$$p_\theta(z \mid x) \;=\; \frac{p_\theta(x \mid z)\, p(z)}{p_\theta(x)}.$$

The catch, which Topic 2 addresses, is that neither the marginal nor the posterior has a closed form when the decoder is a neural network: the integral over z is high-dimensional and cannot be brute-forced.

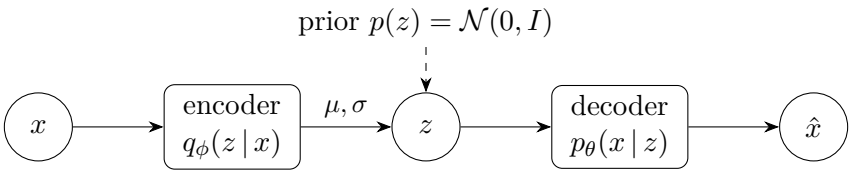


Figure 1: The two networks in a VAE. The encoder maps x to the parameters of a distribution over z; the decoder maps a sample z back to a distribution over x. The dashed arrow is the sampling path used at generation time: draw z from the prior and decode.

1.3 Autoencoders: useful representations, not a generative model

A regular (deterministic) autoencoder looks almost identical: an encoder network compresses x into a low-dimensional code z, and a decoder reconstructs x from z. Training the pair end-to-end with a reconstruction loss (for example, mean squared error) gives a useful representation: the bottleneck z captures the structure the decoder needs in order to rebuild x. Foster covers this construction in Chapter 3 of *Generative Deep Learning* (2nd ed., 2023).

Why is this not a generative model? Three concrete problems.

- **No prior on z.** The encoder is free to place each training point anywhere in the latent space. If you sample a random z at inference time and decode it, you will probably land in a region the encoder never used; the decoder has no idea what to do, and the output is gibberish.
- **No probability over x.** The decoder produces a single reconstruction, not a distribution. You cannot ask "how likely is this x?"
- **Reconstruction-only objective.** Nothing in the loss encourages the latent space to be coherent for sampling.

A vanilla autoencoder is therefore a good *representation learner*, but not a *sampler*. The VAE fixes this with three changes: a prior on z, probabilistic encoder and decoder, and a regularizer that pulls the encoded distributions toward the prior. Those three changes turn the autoencoder into a generative model.

1.4 Representation learning, briefly

Even setting aside generation, latent-variable models are valuable because the latent space is often a much better representation of x for downstream tasks (classification, clustering, similarity search, anomaly detection) than the raw pixels. A good latent space is

- *smooth*: nearby z decode to similar x;
- *structured*: semantic factors of variation correspond to specific directions in z;
- *low-dimensional*: much smaller than $\dim(x)$ without losing the information you care about.

Topic 3 returns to this when we look at latent-space interpolation and arithmetic. See Bengio, Courville, and Vincent, "Representation Learning: A Review and New Perspectives" (IEEE TPAMI, 2013; arXiv:1206.5538) for background on what makes a representation good.

1.5 Payoff

Everything that follows is a way of making the two operations above (generate, infer) tractable when the decoder is a neural network. Topic 2 handles the intractable marginal and posterior and derives the training objective (the ELBO). Topic 3 turns the ELBO into a concrete loss and shows what sampling looks like. Topic 4 covers the practical knobs you need for the coding assignment.

2 Topic 2: Variational Inference and the ELBO

2.1 Plain-English summary

We want to fit a generative model that has a hidden variable inside it. The standard way to train such a model is to maximize the likelihood of the data, but for any interesting decoder the likelihood involves an integral we cannot compute. The fix has two moves: (1) introduce an *approximate* answer to "what hidden state probably produced this data point?" using a neural network called the *encoder*; and (2) replace the impossible likelihood with the **Evidence Lower Bound (ELBO)**, which we *can* compute and which is guaranteed to sit at or below the true log-likelihood. Maximizing the ELBO trains the encoder and decoder together with ordinary gradient descent, with one extra trick (reparameterization) to make random sampling differentiable.

2.2 Overview of this topic

- We begin with the VAE setting: a latent-variable model $p_\theta(x, z) = p(z)\, p_\theta(x \mid z)$ and an encoder $q_\phi(z \mid x)$ that approximates the intractable posterior.
- We derive the ELBO in **two ways**: (A) via **Jensen's inequality**; (B) via a **KL decomposition** that shows the gap to $\log p_\theta(x)$ is $\mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p_\theta(z \mid x)\big)$.
- We clarify the **sign**: ELBO = reconstruction $-\ \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big)$.
- We explain the $\mathrm{KL}(q \,\|\, p)$ notation (the vertical bars are an *ordered-argument separator*, not an absolute value; the order matters because KL is asymmetric).
- We show how Eq. (2) in Kingma and Welling (2014) becomes their Eq. (3) by expanding $\log p_\theta(x, z)$ and recognizing a negative KL term.

2.3 Acronyms and notation

VAE

Variational Autoencoder: latent-variable model with probabilistic encoder $q_\phi(z \mid x)$ and decoder $p_\theta(x \mid z)$ trained by maximizing the ELBO.

ELBO

Evidence Lower Bound: computable lower bound on $\log p_\theta(x)$,

$$\mathcal{L}(\theta, \phi; x) = \mathbb{E}_{q_\phi(z \mid x)}[\log p_\theta(x \mid z)] - \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big).$$

KL

Kullback-Leibler divergence: $\mathrm{KL}(q \,\|\, p) = \int q(z) \log \frac{q(z)}{p(z)}\, dz = \mathbb{E}_q\big[\log \frac{q}{p}\big]$; asymmetric measure of discrepancy between distributions.

SGVB
: **Stochastic Gradient Variational Bayes**: reparameterized Monte Carlo estimator of the ELBO enabling backpropagation.

AEVB
: **Auto-Encoding Variational Bayes**: application of SGVB with neural encoders and decoders (what is now called VAE training).

i.i.d.
: **Independent and identically distributed**: common assumption for datapoints in a dataset.

BCE
: **Binary Cross-Entropy**: negative log-likelihood for Bernoulli observation models.

MSE
: **Mean Squared Error**: corresponds (up to constants) to Gaussian negative log-likelihood with fixed variance.

PDF/CDF
: **Probability Density Function** and **Cumulative Distribution Function**.

RT
: **Reparameterization Trick**: write $z = g_\phi(\varepsilon, x)$ with ε independent of ϕ (e.g., $\varepsilon \sim \mathcal{N}(0, I)$) so gradients can flow.

$\odot$ Elementwise (Hadamard) product.

$\mathcal{N}(\mu, \Sigma)$
: Multivariate normal with mean μ and covariance Σ; $\mathcal{N}(0, I)$ is standard normal.

$\mathrm{diag}(\sigma^2)$
: Diagonal covariance matrix with entries $\sigma_1^2, \ldots, \sigma_k^2$.

x, z
: Observed data x and latent variable z.

$p_\theta(x \mid z)$
: Decoder / likelihood; θ are generative parameters.

$q_\phi(z \mid x)$
: Encoder / variational posterior; ϕ are recognition parameters.

$p(z)$
: Prior over latents (often $\mathcal{N}(0, I)$).

$\mu(x), \sigma(x)$
: Encoder outputs parameterizing $q_\phi(z \mid x)$ (typically diagonal Gaussian).

$\varepsilon \sim \mathcal{N}(0, I)$
: Parameter-free noise used in the reparameterization $z = \mu(x) + \sigma(x) \odot \varepsilon$.

Expectations with respect to a density r are written $\mathbb{E}_r[\cdot]$.

2.4 Prerequisites (quick refresher)

Random variables, densities, expectations

Let Z be a random variable with density $r(z)$. For any integrable function f,

$$\mathbb{E}_{Z\sim r}[f(Z)] \;=\; \int f(z)\, r(z)\, dz.$$

We often shorten to $\mathbb{E}_r[f(Z)]$ or $\mathbb{E}_r[f(z)]$.

Log-likelihoods and nats

We use natural logarithms. Quantities like $\log p(\cdot)$ are in *nats*. (Base-2 logs would measure in *bits*.)

Concavity and Jensen's inequality

A function f is *concave* if for $0 \le \lambda \le 1$, $f(\lambda a + (1-\lambda)b) \ge \lambda f(a) + (1-\lambda) f(b)$. The $\log(\cdot)$ function is concave. **Jensen's inequality** says that for a concave f and a random variable Y,

$$f(\mathbb{E}[Y]) \;\ge\; \mathbb{E}[f(Y)].$$

In particular, for $f = \log$,

$$\log \mathbb{E}[Y] \;\ge\; \mathbb{E}[\log Y].$$

Plain English: a concave function like log "bends downward," so applying log *after* averaging yields a value *no smaller* than averaging the logged values. This lets us swap a hard "log of an expectation" for an easier "expectation of a log," creating a useful lower bound.

Bayes' rule and the posterior

For a joint density $p_\theta(x, z) = p(z)\, p_\theta(x \mid z)$, the posterior is

$$p_\theta(z \mid x) \;=\; \frac{p_\theta(x, z)}{p_\theta(x)} \;=\; \frac{p(z)\, p_\theta(x \mid z)}{\int p(z')\, p_\theta(x \mid z')\, dz'}.$$

In many models, this integral (the *evidence*) is intractable.

Support / absolute continuity (why it matters)

To define $\mathrm{KL}(q \,\|\, p)$ and expectations like $\mathbb{E}_q[\log p]$, we require that wherever $q(z) > 0$, $p(z) > 0$ as well (i.e., q is absolutely continuous w.r.t. p on the support of interest). This avoids division-by-zero and $-\infty$ issues.

2.5 Kullback-Leibler (KL) divergence: definition, notation, intuition

Definition and equivalent forms

Let q and p be densities on the same support. The KL divergence (from q to p) is

$$\mathrm{KL}(q \,\|\, p) \;=\; \int q(z)\, \log \frac{q(z)}{p(z)}\, dz \;=\; \mathbb{E}_q\!\left[\log \frac{q(z)}{p(z)}\right].$$

Meaning of the vertical bars

The notation $\mathrm{KL}(q \,\|\, p)$ is read "KL of q relative to p." **The vertical bars "$\|$" are** *just a separator of ordered arguments.* The first argument (q) supplies the measure of the expectation. *It is not* an absolute value, a norm, or a conditioning operator. The order matters:

$$\mathrm{KL}(q \,\|\, p) \neq \mathrm{KL}(p \,\|\, q) \quad \text{in general.}$$

Core properties (why it is a divergence)

- **Nonnegativity (Gibbs' inequality):** $\mathrm{KL}(q \,\|\, p) \geq 0$, with equality iff $q = p$ almost everywhere.
- **Asymmetry:** $\mathrm{KL}(q \,\|\, p) \neq \mathrm{KL}(p \,\|\, q)$ in general.
- **Link to (cross-)entropy:** $\mathrm{KL}(q \,\|\, p) = H(q, p) - H(q)$, where $H(q) = -\mathbb{E}_q[\log q]$ and $H(q, p) = -\mathbb{E}_q[\log p]$.
- **Coding view (plain English):** $\mathrm{KL}(q \,\|\, p)$ is the *extra code-length* (in nats) you pay on average if data truly come from q but you encode them with a code optimized for p.

The Gaussian closed form used in VAE code, and a worked example of asymmetry, appear in Topic 3.

2.6 The VAE setting and objective

We model data x with latent z:

$$p_\theta(x, z) = p(z)\, p_\theta(x \,|\, z), \qquad p(z) = \mathcal{N}(0, I) \text{ (typical).}$$

The true posterior $p_\theta(z \,|\, x)$ is often intractable. We introduce a tractable *encoder* $q_\phi(z \,|\, x)$ to approximate it and optimize a lower bound on $\log p_\theta(x)$, the *ELBO*.

Variational inference as optimization. The true posterior is the right answer to the question "given that I observed this x, what hidden state z most likely produced it?" We cannot write it down, so we pick a tractable family $q_\phi(z \,|\, x)$ (in practice a diagonal Gaussian whose mean and standard deviation are produced by a neural network with parameters ϕ) and look for the member of that family closest to the true posterior in the KL sense:

$$q_\phi^\star(z \,|\, x) \;=\; \arg\min_{q_\phi} \; \mathrm{KL}\big(q_\phi(z \,|\, x) \,\|\, p_\theta(z \,|\, x)\big).$$

This is the central trade: an integration problem we cannot solve becomes an optimization problem we can. From here on we learn ϕ (encoder) and θ (decoder) jointly by gradient descent.

2.7 ELBO derivation A (via Jensen's inequality)

Step-by-step (math)

1. Evidence (marginal likelihood):

$$\log p_\theta(x) = \log \int p_\theta(x, z)\, dz.$$

2. Insert $q_\phi(z \mid x)$ (change of measure):

$$\log p_\theta(x) = \log \int q_\phi(z \mid x) \, \frac{p_\theta(x,z)}{q_\phi(z \mid x)} \, dz = \log \mathbb{E}_{q_\phi}\left[\frac{p_\theta(x,z)}{q_\phi(z \mid x)}\right].$$

3. Concavity of log (Jensen):

$$\log p_\theta(x) \;\geq\; \mathbb{E}_{q_\phi}[\log p_\theta(x,z) - \log q_\phi(z \mid x)] \, .$$

4. Expand the joint:

$$\log p_\theta(x,z) = \log p_\theta(x \mid z) + \log p(z).$$

5. Collect terms:

$$\boxed{\mathcal{L}(\theta,\phi;x) = \mathbb{E}_{q_\phi}[\log p_\theta(x \mid z)] - \mathrm{KL}(q_\phi(z \mid x) \,\|\, p(z)) \;\leq\; \log p_\theta(x)}$$

Step-by-step (plain English)

1. We want $\log p_\theta(x)$ but integrating over z is hard.
2. Rewrite that integral as an expectation under a distribution we control (the encoder q_ϕ).
3. Use the "log bends downward" trick to turn a hard log of an average into an easier average of a log, producing a lower bound.
4. Split the joint into likelihood and prior pieces.
5. The bound is "**reconstruction** minus **regularizer**"; the regularizer keeps $q_\phi(z \mid x)$ near $p(z)$.

When is it tight? Equality in Jensen holds iff $\frac{p_\theta(x,z)}{q_\phi(z \mid x)}$ is (a.s.) constant in z, i.e. when $q_\phi(z \mid x) = p_\theta(z \mid x)$.

2.8 ELBO derivation B (KL decomposition / "gap" identity)

Step-by-step (math)

1. Start with a nonnegative divergence:

$$\mathrm{KL}(q_\phi(z \mid x) \,\|\, p_\theta(z \mid x)) \;\geq\; 0.$$

2. Bayes: $p_\theta(z \mid x) = \frac{p_\theta(x,z)}{p_\theta(x)}$:

$$\mathrm{KL} = \mathbb{E}_{q_\phi}\big[\log q_\phi(z \mid x) - \log p_\theta(x,z) + \log p_\theta(x)\big].$$

3. Rearranging gives

$$\boxed{\log p_\theta(x) = \underbrace{\mathbb{E}_{q_\phi}\big[\log p_\theta(x,z) - \log q_\phi(z \mid x)\big]}_{\text{ELBO}} + \mathrm{KL}(q_\phi(z \mid x) \,\|\, p_\theta(z \mid x))} \, .$$

4. Expanding the joint recovers the same two-term ELBO as in Derivation A.

Step-by-step (plain English)

1. KL is never negative, so it can quantify a "gap to optimality."
2. Bayes' rule inserts $\log p_\theta(x)$ as a constant inside the expectation.
3. Rearrangement shows: $\log p_\theta(x) = \text{ELBO} + \text{gap}$, therefore ELBO is a lower bound.
4. If q_ϕ equals the true posterior, the gap vanishes and the bound is exact.

What the identity buys us. The right-hand side has one piece we can compute (the ELBO) and one we cannot (the KL to the true posterior). If we push the ELBO up by changing θ and ϕ, the identity forces one of two things to happen: either $\log p_\theta(x)$ goes up (the model fits the data better) or the KL gap shrinks (the approximate posterior gets closer to the true one). Both are good, and both come from the same gradient step.

2.9 How Eq. (2) becomes Eq. (3) in Kingma and Welling

Starting from (their Eq. (2))

$$\mathcal{L}(\theta, \phi; x) = \mathbb{E}_{q_\phi(z \mid x)}\big[-\log q_\phi(z \mid x) + \log p_\theta(x, z)\big],$$

expand $\log p_\theta(x, z) = \log p_\theta(x \mid z) + \log p(z)$ and regroup:

$$\mathcal{L}(\theta, \phi; x) = \mathbb{E}_{q_\phi}[\log p_\theta(x \mid z)] + \mathbb{E}_{q_\phi}[\log p(z) - \log q_\phi(z \mid x)] = \mathbb{E}_{q_\phi}[\log p_\theta(x \mid z)] - \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big),$$

which is their Eq. (3).

Plain English. Replace the joint by likelihood + prior, then recognize the second expectation as a *negative* KL because $\mathrm{KL}(q \,\|\, p) = \mathbb{E}_q[\log q - \log p]$.

Read the result as a tug-of-war. The first term asks: when the encoder turns x into a distribution over z, and the decoder turns a sampled z back into a distribution over x, how well do we recover the original x? The second term asks: is the distribution the encoder produced for this x close to the prior we will sample from later? The two terms genuinely fight. Reconstruction alone would let the encoder give each training point its own private corner of latent space (great reconstruction, useless for sampling); the KL alone would let the encoder collapse onto the prior and ignore x (great regularization, nothing left for the decoder). Topic 3 makes this concrete.

2.10 Reparameterization trick (why we can backprop through sampling)

The ELBO is an expectation over $q_\phi(z \mid x)$, which depends on the encoder parameters ϕ. If we drew z directly, the gradient with respect to ϕ would have to pass through a random sampling step, which has no derivative. Suppose instead that $z \sim q_\phi(z \mid x)$ can be written as a differentiable transformation of base noise:

$$z \;=\; g_\phi(\varepsilon, x), \qquad \varepsilon \sim p(\varepsilon).$$

Gaussian case (common in VAEs):

$$z \;=\; \mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I).$$

Then for any f,

$$\mathbb{E}_{q_\phi(z\,|\,x)}[f(z)] = \mathbb{E}_{\varepsilon\sim p(\varepsilon)}[f(g_\phi(\varepsilon, x))],$$

and the right-hand side is differentiable w.r.t. ϕ: gradients ∇_ϕ flow through the deterministic map g_ϕ.

Plain English. Move randomness outside the network into a fixed noise source; make the rest deterministic so autodiff works end-to-end. Instead of "sample z from a distribution whose parameters depend on ϕ," we say "draw a fixed-distribution ε once, then transform it through learnable knobs." Same result statistically, but now it is differentiable.

2.11 Sign conventions and what to optimize

The training objective is to *maximize* the ELBO

$$\mathcal{L}(\theta, \phi; x) = \mathbb{E}_{q_\phi}[\log p_\theta(x\,|\,z)] - \mathrm{KL}\big(q_\phi(z\,|\,x)\,\|\,p(z)\big),$$

or equivalently *minimize* $-\mathcal{L}$, which is "reconstruction loss + KL penalty." The "+ KL" you sometimes see appears in the identity

$$\log p_\theta(x) = \mathrm{ELBO} + \mathrm{KL}\big(q_\phi(z\,|\,x)\,\|\,p_\theta(z\,|\,x)\big),$$

where the KL is the nonnegative *gap* to the true log evidence, not part of the ELBO.

2.12 Key takeaways

- ELBO is a computable lower bound on $\log p_\theta(x)$: $\mathcal{L} = \mathbb{E}_{q_\phi}[\log p_\theta(x\,|\,z)] - \mathrm{KL}(q_\phi(z\,|\,x)\,\|\,p(z))$.
- Two derivations: **Jensen** and **KL-gap identity**. The "+ KL" you sometimes see is the *gap*, not part of ELBO.
- $\mathrm{KL}(q\,\|\,p)$ is an *ordered* divergence; "$\|$" is just an argument separator.
- Reparameterization lets gradients flow through samples: $z = \mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon$.
- The **encoder** implements $q_\phi(z\,|\,x)$; the **decoder** implements $p_\theta(x\,|\,z)$. Sampling new data is: draw $z \sim \mathcal{N}(0, I)$, then decode.

Pointer to the original source: D. P. Kingma and M. Welling, *Auto-Encoding Variational Bayes*, ICLR 2014 (arXiv:1312.6114). A tutorial-length treatment is Kingma and Welling, *An Introduction to Variational Autoencoders*, 2019 (arXiv:1906.02691).

3 Topic 3: Reconstruction Loss, KL Divergence, and Sampling

3.1 Plain-English summary

Topic 2 derived the ELBO as a single mathematical object. To actually *train* a VAE we need to turn it into code: a loss function with two easy-to-compute pieces, a way to sample from the trained model, and a few diagnostics to check that training is healthy. By the end of this topic you should be able to read the loss function in a standard VAE notebook (for example, Foster's Fashion-MNIST VAE) and explain every line.

3.2 From ELBO to a practical loss

The training objective (*maximize* ELBO) is equivalent to *minimizing*

$$\mathcal{L}_{\text{VAE}}(\theta, \phi; x) \;=\; -\mathcal{L}(\theta, \phi; x) = \underbrace{-\mathbb{E}_{q_\phi}[\log p_\theta(x \mid z)]}_{\text{reconstruction loss}} + \underbrace{\text{KL}(q_\phi(z \mid x) \,\|\, p(z))}_{\text{KL regularizer}}.$$

Both pieces have tractable expressions for the choices made below. That is the whole reason we set up the math the way we did.

The reconstruction term in practice

The expectation $\mathbb{E}_{q_\phi(z \mid x)}[\log p_\theta(x \mid z)]$ is approximated by drawing a single sample $z = \mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon$ (the reparameterization trick) and evaluating $\log p_\theta(x \mid z)$ on that sample. One sample per datapoint is enough in practice because we average over a minibatch.

Likelihood choices and the corresponding reconstruction term:

- *Bernoulli decoder* (binary or $[0, 1]$-valued x): $-\mathbb{E}_{q_\phi}[\log p_\theta(x \mid z)]$ is the expected **binary cross-entropy** (BCE), summed over pixels. This is the form used for binarized image data such as Fashion-MNIST.
- *Gaussian decoder* (real x with fixed variance σ^2): $-\mathbb{E}_{q_\phi}[\log p_\theta(x \mid z)]$ is (up to constants) proportional to an expected **mean-squared error** (MSE). This is what most photographic-image VAEs use.

The decoder architecture is the same in both cases; only the final activation (sigmoid vs. linear) and the loss change.

3.3 The KL term in closed form

Gaussian closed form (used in VAEs)

If $q_\phi(z \mid x) = \mathcal{N}(\mu, \text{diag}(\sigma^2))$ in $\mathbb{R}^k$ and $p(z) = \mathcal{N}(0, I)$,

$$\text{KL}(q_\phi(z \mid x) \,\|\, p(z)) \;=\; \tfrac{1}{2} \sum_{i=1}^{k} \Big(\mu_i^2 + \sigma_i^2 - \log \sigma_i^2 - 1\Big).$$

Plain English: penalizes means far from 0 and variances that are too large *or* too small, keeping posteriors smooth and near the prior. This is a single line of code per datapoint. Note that the formula is what fixes the prior to $\mathcal{N}(0, I)$ in most implementations; swapping the prior requires a different (and usually less convenient) KL expression.

Mini example: KL asymmetry with 1D Gaussians

Let $q = \mathcal{N}(\mu_q, \sigma_q^2)$, $p = \mathcal{N}(\mu_p, \sigma_p^2)$. Then

$$\text{KL}(q \,\|\, p) \;=\; \frac{1}{2}\left[\frac{\sigma_q^2}{\sigma_p^2} + \frac{(\mu_p - \mu_q)^2}{\sigma_p^2} - 1 + \log \frac{\sigma_p^2}{\sigma_q^2}\right], \quad \text{KL}(p \,\|\, q) \;\text{ swaps } (\mu_q, \sigma_q^2) \text{ and } (\mu_p, \sigma_p^2).$$

Unless $\mu_q = \mu_p$ and $\sigma_q^2 = \sigma_p^2$, the two numbers differ, illustrating **asymmetry**. Setting $\mu_p = 0$, $\sigma_p^2 = 1$ recovers one term of the VAE closed form above.

3.4 The reconstruction-vs-regularization tug-of-war, made concrete

In code you literally watch two scalars compete during training.

- **Reconstruction low, KL high.** The encoder is putting each x far from the prior to give the decoder more capacity. Reconstructions look great; prior samples look terrible.
- **KL near zero, reconstruction high.** The encoder has collapsed $q_\phi(z \mid x)$ onto the prior and is ignoring x. This is **posterior collapse**. The prior is well matched; reconstructions are blurry blobs.

Practical tips and common pitfalls

- **β-VAE** (Higgins et al., ICLR 2017): replace the KL term by $\beta \cdot \mathrm{KL}(\cdot)$,

$$\mathcal{L}_\beta(\theta, \phi; x) \;=\; -\mathbb{E}_{q_\phi}[\log p_\theta(x \mid z)] + \beta\, \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big),$$

  to enforce disentanglement ($\beta > 1$) or boost reconstruction ($\beta < 1$). Standard VAE code implicitly uses $\beta = 1$; changing it is one line.
- **KL annealing / warm-up**: gradually increase the KL weight from 0 to 1 over the first few epochs to avoid *posterior collapse* (encoder ignored by a strong decoder). Bowman et al., "Generating Sentences from a Continuous Space" (CoNLL 2016; arXiv:1511.06349) is the classic worked example in a text-VAE setting.
- **Aggregated posterior vs. prior**: even if each $q_\phi(z \mid x)$ is close to $p(z)$, the mixture $q(z) = \int q_\phi(z \mid x) p_{\text{data}}(x)\, dx$ can deviate; richer priors (e.g., VampPrior, flows) can help.
- **Units**: logs in natural base $\Rightarrow$ losses in *nats*. Switching bases scales constants but not optima.

3.5 Sampling: how a trained VAE generates new data

Once training is done, generation is two lines:

1. Draw $z \sim \mathcal{N}(0, I)$ from the prior.
2. Decode: produce $\hat{x}$ from $p_\theta(x \mid z)$. For a Bernoulli decoder the per-pixel probability is your image; for a Gaussian decoder the mean is your image.

That this works at all is the payoff for the KL regularizer: it forces the encoder to use latent positions that are also plausible under the prior, so sampling from the prior lands in regions the decoder knows what to do with. Without the KL term (a vanilla autoencoder), step 2 would produce gibberish.

3.6 Latent-space structure: interpolation and arithmetic

A well-trained VAE has a latent space that is smooth and semantically organized. Two visualizations, both easy to reproduce on Fashion-MNIST or a face dataset:

- **Linear interpolation.** Pick two real images x_1, x_2; encode to get z_1, z_2 (use the means μ_ϕ). Walk along

$$z(t) = (1 - t)\, z_1 + t\, z_2, \qquad t \in [0, 1],$$

  and decode at each step. A trained VAE produces a smooth morph, not a sequence of glitches, because nearby z decode to similar x.

- **Latent arithmetic.** Average the latent codes of all training points with attribute A (for example, "smiling") and subtract the average code of all points without it. The resulting vector behaves like a direction for that attribute: add it to any encoded z and the decoded image gains the attribute. Foster demonstrates this in Chapter 3.

3.7 Payoff

The ELBO gives us a loss with two named, computable terms; the closed-form KL and the reparameterized reconstruction make it a few lines of code; and the KL regularizer is exactly what makes "sample from the prior and decode" produce sensible outputs. When training misbehaves, the two loss scalars tell you which side of the tug-of-war is winning, and β or KL warm-up let you rebalance it.

4 Topic 4: Practical Considerations When Training and Evaluating a VAE

4.1 Plain-English summary

Topics 1 through 3 told you what a VAE is and what it optimizes. This topic is the "before you open the notebook" reference: the architecture choices you will face, the training knobs that actually matter, the failure modes to recognize, the evaluation evidence to trust, and the compute footprint of the assignment.

4.2 Architecture choices

The encoder and decoder are independent design problems, but they share a few rules of thumb.

- **Latent dimension d.** The single most important hyperparameter. Too small and the decoder cannot reconstruct meaningful detail; too large and the KL regularizer has trouble keeping the posterior compact. Common starting points: $d = 2$ for visualization, $d = 10$ to 50 for Fashion-MNIST, $d = 100$ to 200 for face datasets.

- **Encoder network.** For images, a few convolutional layers with stride-2 downsampling, ending in two parallel fully-connected heads that output $\mu_\phi(x)$ and $\log \sigma^2_\phi(x)$. Outputting the *log*-variance keeps the value unconstrained and numerically stable:

$$\sigma_\phi(x) = \exp(\tfrac{1}{2} \log \sigma^2_\phi(x)), \qquad z = \mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon.$$

- **Decoder network.** Mirror image of the encoder: fully-connected to a small spatial volume, then transposed convolutions (or upsampling plus regular convolutions) up to the input resolution. Final activation is sigmoid for $[0, 1]$ pixel data (Bernoulli decoder) or linear for general-range data (Gaussian decoder).

- **Symmetry.** Encoder and decoder do not strictly need to be symmetric, but starting symmetric is a sensible default.

4.3 Training knobs that matter

- **Optimizer:** Adam with learning rate around 10^{-3} or $5 \cdot 10^{-4}$. Almost never the bottleneck.

- **Batch size:** 64 to 256 is fine for Fashion-MNIST on a laptop. Larger batches give more stable KL estimates but cost memory.
- **KL warm-up:** ramp the KL coefficient from 0 to 1 over the first 5 to 20 epochs. Helps avoid posterior collapse, especially when d is large.
- **Number of epochs:** 20 to 50 for Fashion-MNIST is usually enough to see a clear pattern; longer is rarely transformative.
- **Reproducibility:** set a fixed random seed for NumPy, the framework (TensorFlow or PyTorch), and the dataloader. Log the seed in your write-up.

4.4 Failure modes to recognize

- **Posterior collapse.** KL loss collapses to (near) zero early in training. Symptom: the encoder ignores x, so reconstructions look like a single average image regardless of input. Remedies: KL warm-up, smaller β, more capacity in the decoder bottleneck.
- **Blurry reconstructions.** The classic VAE failure mode. Most often a consequence of the Gaussian decoder assumption (the mean of a multimodal distribution *is* blurry). Often acceptable; not the same problem as posterior collapse.
- **Exploding variance.** If $\log \sigma_\phi^2$ is not bounded, training can drift into unreasonably large variances. Clipping the log-variance to, say, $[-5, 5]$ is a simple guard.
- **Mode dropping.** Samples cover only some of the modes of the data distribution. Less common in VAEs than in GANs, but watch for it in your prior-sample grids.

4.5 Evaluation: how do you know it worked?

VAE evaluation is genuinely hard, because no single number captures "sample quality." Combine three kinds of evidence.

1. **Loss curves.** Plot total loss, reconstruction loss, and KL loss separately as a function of epoch. Both terms should decrease at first; KL typically rises slightly later as the model trades off. Big gaps or single-term collapses are red flags.
2. **Sample quality.** Draw a grid (e.g., 16 samples) from the prior and decode. Inspect by eye. For Fashion-MNIST you should recognize garment classes after a few epochs.
3. **Latent-space structure.** Encode the test set, take the means $\mu_\phi(x)$, and either scatter directly (if $d = 2$) or project with t-SNE or UMAP. Color points by class label. A well-trained VAE shows class clusters even though it was trained without labels.

Quantitative metrics such as FID (Fréchet Inception Distance) and IS (Inception Score) exist but are most useful for comparing methods on the same benchmark; they are overkill for the Module 2 assignment.

4.6 Compute footprint

The Fashion-MNIST VAE notebook trains in roughly 5 to 15 minutes on a CPU laptop. You do not need a GPU for the required assignment. If your machine struggles, the university HPC cluster via Open OnDemand JupyterLab (covered in Module 1) is the recommended fallback. The stretch face-dataset notebook benefits noticeably from a GPU.

4.7 Payoff

You are now ready for the Module 2 VAE Technical Assignment: run the Fashion-MNIST VAE, vary the latent dimension, sample from the prior, visualize the latent space, and write a short interpretation. Topic 3 gave you the loss-function vocabulary; this topic gave you the diagnostics and the architecture defaults. If you can read the two loss scalars, look at a prior-sample grid, and look at a class-colored latent scatter, you can diagnose most of what goes wrong with a VAE.

References and further reading

- D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," ICLR 2014. arXiv:1312.6114.
- D. P. Kingma and M. Welling, *An Introduction to Variational Autoencoders*, 2019. arXiv:1906.02691.
- C. Doersch, "Tutorial on Variational Autoencoders," 2016. arXiv:1606.05908.
- I. Higgins et al., "β-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework," ICLR 2017.
- S. R. Bowman et al., "Generating Sentences from a Continuous Space," CoNLL 2016. arXiv:1511.06349.
- D. Rezende and S. Mohamed, "Variational Inference with Normalizing Flows," ICML 2015. arXiv:1505.05770.
- D. Kingma et al., "Improved Variational Inference with Inverse Autoregressive Flow," NeurIPS 2016. arXiv:1606.04934.
- Y. Bengio, A. Courville, and P. Vincent, "Representation Learning: A Review and New Perspectives," IEEE TPAMI, 2013. arXiv:1206.5538.
- I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016, Chapter 14 (Autoencoders).
- D. Foster, *Generative Deep Learning*, 2nd ed., O'Reilly, 2023, Chapter 3 (Variational Autoencoders). Companion notebooks: https://github.com/davidADSP/Generative_Deep_Learning_2nd_Edition.